The Universal Software Engineering Constitution System (USECS) v2.0

Preamble

This constitution defines the universal engineering, UI/UX, architecture, scalability, accessibility, security, and consistency laws that all AI agents and developers must follow while building websites or web applications.

Its purpose is to:

- eliminate design drift
- reduce hallucinated implementations
- prevent inconsistent architecture
- ensure scalable systems
- enforce predictable UI/UX behavior
- create maintainable production-ready applications
- standardize AI-generated development workflows

Rule Severity System

Every rule in this constitution is tagged with one of three severity levels.

Level	Meaning
[CRITICAL]	Security, accessibility, data integrity, infrastructure safety. Must never be ignored under any circumstance.
[RECOMMENDED]	Best-practice architectural guidance. May be adapted depending on project complexity.
[OPTIONAL]	Project-dependent enhancements. Not all systems require every pattern listed here.

> Engineering complexity must remain proportional to product complexity. Do not apply enterprise abstractions to landing pages or MVPs.

Foundational Directives

Directive 1 — Think Before Generating

No AI agent or developer shall directly begin coding.

Before implementation begins, the following must be defined and locked:

1. All pages and routes
2. User flows
3. Reusable components

4. Layouts
5. API architecture
6. State management strategy
7. Folder structure
8. Validation rules
9. Responsive strategy
10. Accessibility requirements
11. Security requirements
12. SEO requirements
13. Deployment strategy

> **Implementation without planning is prohibited.**

Directive 2 — Reusability Before Duplication

If logic, styling, or structure repeats more than twice:

- create a reusable component
- create a utility/helper
- create a shared hook
- create a common validator
- create a shared configuration

Duplicate logic must be minimized.

> **Golden Rule: Repetition is a signal of missing architecture.**

Directive 3 — Predictability Over Cleverness

Systems must prioritize:

- readability
- maintainability
- explicit logic
- predictable behaviors
- deterministic architecture

Avoid:

- hidden side effects
- magic numbers
- deeply nested conditions
- unclear abstractions
- overengineered logic

Clarity is always preferred over complexity.

Article I — Architectural Blueprinting

Before writing any code, the blueprint must be generated and locked.

Section 1 — Sitemap & Route Declaration

Explicitly define all pages and routes before writing code, including:

- public and private routes
- role-based routes
- fallback routes
- error pages

```
/
/login
/register
/dashboard
/settings
/profile
/admin
404 — Not Found
403 — Forbidden
```

Section 2 — Component Matrix

Every page must document all components used, distinguishing reusable shared components from page-specific ones.

```
Dashboard Page
  Shared:  Sidebar, TopNavbar, AnalyticsCard
  Page:    RevenueChart, RecentActivityTable
```

All reusable components must reside in:

```
src/components/shared/
```

Never duplicate shared components across page directories.

Section 3 — Layout Constitution

Master layout wrappers must exist for every context. All pages must inherit layouts instead of recreating structural shells.

```
MainLayout      — Public pages
DashboardLayout — Authenticated views
AuthLayout      — Login / Register
AdminLayout     — Admin panel
```

This guarantees consistent headers, footers, spacing, navigation, and responsiveness.

Article II — Project Structure Constitution

Section 1 — Folder Structure

All projects must follow structured organization. Random file placement is prohibited.

```
src/
 ├── api/          — Centralized API layer
 ├── assets/       — Static files
 ├── components/   — Shared and page components
 ├── config/       — App-wide configuration
 ├── constants/    — Named constants only
 ├── hooks/        — Custom React hooks
 ├── layouts/      — Layout wrappers
 ├── pages/        — Route-level page components
 ├── routes/       — Route definitions and guards
 ├── services/     — Business logic and API calls
 ├── store/        — Global state
 ├── styles/       — Design tokens and globals
 ├── tests/        — Unit and integration tests
 ├── types/        — TypeScript types and interfaces
 ├── utils/        — Pure utility functions
 └── validators/   — Validation schemas
```

Section 2 — Naming Convention Constitution

Element	Convention	Example
Components	PascalCase	`UserProfileCard.tsx`
Functions	camelCase	`fetchUserData()`
Constants	UPPER_SNAKE_CASE	`MAX_UPLOAD_SIZE`
CSS classes	kebab-case / BEM / tokens	`user-profile-card`
Files (non-component)	camelCase	`authService.ts`

Avoid unclear naming.

```
Bad:  box2finalnew
Good: user-profile-card
```

Article III — The Global Design System

Hardcoded styling values inside components are forbidden unless technically unavoidable.

All styling must reference:

- design tokens
- theme variables
- global spacing systems
- centralized typography rules

Section 1 — Color Palette Dictionary

Define a full color dictionary globally.

Category	Token	Usage
Brand	`--color-primary`	CTAs, links, key actions
Brand	`--color-secondary`	Supporting visual elements
Brand	`--color-accent`	Highlights, badges
Semantic	`--color-success`	Approvals, completions
Semantic	`--color-warning`	Caution, incomplete actions
Semantic	`--color-error`	Failures, destructive actions
Semantic	`--color-info`	Neutral notifications
Text	`--color-text-primary`	Main content
Text	`--color-text-secondary`	Supporting content
Text	`--color-text-muted`	Captions, metadata
Text	`--color-text-disabled`	Inactive states
Surface	`--color-background`	Page background
Surface	`--color-surface`	Card / panel backgrounds
Surface	`--color-border`	Borders and dividers

Section 2 — Typography Constitution

Lock typography globally. Define:

- font families
- heading scales (H1 through H6)
- paragraph styles
- button text styles
- line heights
- letter spacing

Typography inconsistency across components is prohibited.

Section 3 — Spacing Constitution

A universal spacing grid must exist. All margins and paddings must follow this scale. Random spacing values are forbidden.

```
Spacing scale (px): 4, 8, 12, 16, 24, 32, 48, 64, 96
```

Section 4 — Geometry Constitution

Define globally as named tokens:

```
radius-sm:   4px     shadow-sm:  0 1px 3px rgba(0,0,0,0.10)
radius-md:   8px     shadow-md:  0 4px 12px rgba(0,0,0,0.12)
radius-lg:   16px    shadow-lg:  0 8px 24px rgba(0,0,0,0.15)
radius-pill: 9999px
```

Article IV — UI/UX Consistency Constitution

Section 1 — Responsive Design

Every website must support:

- mobile
- tablet
- desktop
- ultrawide displays

Rules:

- mobile-first preferred
- avoid fixed widths
- avoid overflow breakage

- maintain touch-friendly spacing
- maintain readable typography at all breakpoints

Section 2 — Accessibility Constitution **[CRITICAL]**

Accessibility is mandatory and non-negotiable. All applications must:

- support keyboard navigation
- support screen readers
- support tab navigation
- maintain visible focus states
- use semantic HTML throughout
- maintain WCAG AA contrast ratios
- use alt text for all images
- associate labels with all inputs

> **Visual aesthetics must never come at the cost of accessibility compliance.**

Section 3 — Theme Constitution

When dark and light themes are implemented:

- both themes must preserve readability
- maintain identical spacing systems
- preserve component structure
- preserve interaction consistency

Avoid pure black (`#000000`) backgrounds and pure white (`#FFFFFF`) glare-heavy surfaces. Use near-black and near-white values instead.

Article V — Universal Component Standards

Section 1 — Forms & Input Fields

Validation Constitution

Every input must have:

- validation rules
- labels
- placeholders where appropriate
- focus states
- disabled states
- ARIA accessibility labels

Validation categories:

- names: alphabetic validation
- usernames: alphanumeric validation
- emails: RFC-compliant email validation
- passwords: strength-scored validation

Error State Constitution

When validation fails:

- border switches to semantic error color
- descriptive error text appears below the field
- focus shifts to invalid input where appropriate

```
Bad:  'Invalid field'
Good: 'Password must contain at least 1 number and 1 uppercase letter.'
```

Form UX Constitution

Forms must:

- preserve entered values on validation errors
- avoid unnecessary field resets
- support keyboard submission
- prevent duplicate submissions during async processing
- display loading states during active API calls

Section 2 — Button Constitution

All buttons must maintain:

- same typography
- same spacing logic
- same border radius system
- same interaction animations
- same hover philosophy

Button Variant System

Variant	Used For	Visual Style
Primary	Submit, Save, Approve, Continue	Solid fill, primary brand color
Secondary	Cancel, Back, Dismiss	Outlined or ghost style
Destructive	Delete, Remove, Reset	Semantic error color
Ghost	Low-priority supplemental actions	Transparent, subtle hover

Button State Constitution

Every button must support:

- hover state
- active state
- disabled state
- loading state (with double-click prevention)
- focus-visible state

Section 3 — Cards & Containers

Cards may differ in structure by use case but must preserve:

- theme consistency
- spacing consistency
- shadow consistency
- typography consistency
- border radius consistency

Section 4 — Modal/Dialog Constitution **[CRITICAL]**

All modals must:

- use overlays
- trap keyboard focus
- support ESC-to-close
- prevent background interaction
- support outside-click dismissal where appropriate

Modal systems must maintain:

- consistent animation
- consistent spacing
- consistent button alignment
- consistent close behavior

Section 5 — Dropdown Constitution

Dropdowns must:

- support keyboard navigation
- close on outside click
- support active and disabled states
- preserve animation consistency

Section 6 — Checkbox & Radio Constitution

All checkbox/radio systems must maintain:

- consistent size, colors, states, and animations
- clickable labels
- visible focus states
- working keyboard navigation

Article VI — Notification & Feedback Systems

Section 1 — Notification Constitution

A reusable notification system is mandatory.

Category	Semantic Color
Success	Green
Warning	Yellow / Orange
Error	Red
Info	Blue

Notifications must:

- support dismissing
- maintain consistent placement and animation
- support screen readers
- avoid spam under rapid user actions

Section 2 — Loading State Constitution

Every async process must display a visual indicator:

- spinner
- skeleton loader
- shimmer
- progress indicator

Users must never remain uncertain about system activity.

Section 3 — Empty State Constitution

When no data exists:

- explain the situation clearly
- suggest next actions
- maintain visual quality

Blank empty screens are prohibited.

Section 4 — Error State Constitution

Errors must explain what failed and how to recover without exposing raw technical messages.

```
Bad:  '500 Internal Server Error'
Good: 'Unable to load your data. Please check your connection and try again.'
```

Article VII — State Management Constitution

Local state should manage:

- component-specific UI
- temporary interactions

Global state should manage:

- authentication
- themes
- user session
- shared application data

Unnecessary global state is prohibited. All state mutations must remain traceable, predictable, and debuggable. Hidden state mutations are forbidden.

Article VIII — API & Backend Interaction Constitution

Direct API calls inside random UI components are strongly discouraged. All networking logic must be centralized in `services/` or `api/`.

All API integrations must handle:

- loading states
- centralized error handling
- retry logic where appropriate
- request timeouts
- token refresh flows

Frontend systems must never blindly trust backend responses. Validate null values, undefined properties, incorrect types, and missing fields before use. Defensive programming is mandatory.

Article IX — Security Constitution **[CRITICAL]**

Section 1 — Input Security

All user-generated input must be sanitized. Protect against:

- XSS
- injection attacks
- malicious file uploads
- unsafe query parameters

Section 2 — Authentication Constitution

Authentication systems must support:

- secure token handling
- protected route enforcement
- role-based authorization
- session expiration handling
- logout invalidation

Tokens must never be stored in insecure locations.

Section 3 — Secret Management **[CRITICAL]**

Secrets, API keys, and credentials must never appear in:

- frontend code
- repositories
- logs
- build outputs

Environment variables are mandatory. Accidental secret exposure in version control is a critical violation.

Article X — Performance & Reliability Constitution

Section 1 — Performance Budgets

Every production system should define measurable performance budgets. Performance optimization must begin during architecture planning, not after production issues emerge.

Metric	Target
Initial JS Bundle	Under 300KB preferred
Largest Contentful Paint (LCP)	Under 2.5 seconds
Cumulative Layout Shift (CLS)	Under 0.1

Metric	Target
API Response Time	Under 500ms target
Time To Interactive (TTI)	Minimized

Section 2 — Lazy Loading Constitution

Lazy loading must be applied to:

- routes
- large images
- heavy components
- large libraries
- non-critical scripts

Section 3 — Render Optimization

Use when measurably beneficial:

- memoization
- virtualization
- caching
- optimized selectors
- request batching

Avoid premature optimization.

Section 4 — Graceful Degradation Constitution

Systems must assume failures will occur. Applications must support:

- fallback UI
- retry systems
- offline handling where appropriate
- partial rendering
- degraded feature support
- timeout recovery

Section 5 — Observability Constitution

Production systems must support:

- centralized logging
- error tracking
- tracing
- analytics

- monitoring dashboards
- health checks

Monitoring categories:

- frontend errors
- backend failures
- API latency
- database performance
- authentication failures
- infrastructure health

> **Undebuggable systems are operational liabilities.**

Article XI — Database & Data Architecture Constitution

Section 1 — Schema Governance

Database schemas must maintain:

- naming consistency
- normalization where appropriate
- indexing strategy
- relational integrity
- migration tracking

Avoid chaotic schema growth.

Section 2 — Migration Constitution

Schema changes must:

- be versioned
- be reversible where possible
- avoid destructive changes without backups
- support rollback planning

Section 3 — Data Integrity Constitution

Systems must protect:

- transactional integrity
- referential integrity
- concurrency consistency
- auditability

Critical systems should support audit logs.

Section 4 — Soft Delete Constitution

For important business systems where recovery may be needed, soft-delete patterns are preferred over hard deletion. Hard deletion must be intentional and explicitly justified.

Article XII — DevOps & CI/CD Constitution

Every pull request must automatically validate:

- linting
- formatting
- type checking
- tests
- build generation
- dependency vulnerabilities

Broken builds must not reach production.

Deployment pipelines must support:

- rollback systems
- environment separation (development, staging, production)
- staging validation
- production safeguards
- secret management

Environment variable leakage between stages is prohibited.

Article XIII — Dependency Governance Constitution

Before installing any dependency, evaluate:

- maintenance activity
- security reputation
- ecosystem stability
- bundle size impact
- long-term viability

Avoid unnecessary dependency inflation. Never install multiple libraries solving identical problems without strong justification.

```
Avoid: moment + dayjs + date-fns simultaneously without justification.
Choose one date library and standardize across the project.
```

Dependencies must be periodically audited for vulnerabilities, abandonment, and deprecated APIs.

Article XIV — Versioning & Architecture Documentation

Projects must follow semantic versioning:

```
MAJOR.MINOR.PATCH

MAJOR — breaking changes
MINOR — new backward-compatible features
PATCH — fixes and small improvements
```

Major architectural decisions must be captured in Architecture Decision Records (ADRs) explaining not just what was chosen, but why.

```
ADR-001: Why PostgreSQL was selected over MongoDB
ADR-002: Why SSR was implemented instead of CSR
ADR-003: Why Redux Toolkit was chosen for global state
```

Future developers and AI agents must be able to understand the reasoning behind architectural decisions.

Article XV — Human Experience & UX Psychology Constitution

Section 1 — Cognitive Load Reduction

Interfaces must minimize:

- confusion
- excessive choices
- visual clutter
- inconsistent interactions

Section 2 — Visual Hierarchy Constitution

Important actions must visually dominate secondary actions.

- Primary CTA more prominent than secondary CTA
- Dangerous actions visually isolated from safe ones

Section 3 — Perceived Performance Constitution

Systems should feel responsive even during heavy operations. Use:

- optimistic UI
- skeleton loaders
- progress indicators
- smooth transitions

Perceived speed matters as much as actual speed.

Article XVI — Code Quality Constitution

Section 1 — Clean Code Standards

Code must remain:

- readable
- modular
- typed where possible
- maintainable
- scalable

Section 2 — Component Size Limits

Large components must be decomposed into:

- UI sections
- hooks
- utilities
- child components

Massive monolithic files are strongly discouraged.

Section 3 — Comment Constitution

Comments should explain WHY, not describe obvious WHAT.

```
// Bad:  increment counter
counter++

// Good: Prevent duplicate submissions during API retry window.
counter++
```

Article XVII — Testing Constitution

Critical systems must be tested. Minimum required coverage:

- validation logic tests
- API integration tests
- authentication flow tests
- critical user journey tests
- edge case handling

Loading states and error states must also be covered by tests. Untested critical paths are a liability in production.

Article XVIII — SEO Constitution

Public-facing websites must support:

- semantic HTML throughout
- proper metadata tags
- OpenGraph and Twitter card tags
- `sitemap.xml`
- `robots.txt`
- canonical URLs
- optimized heading hierarchy (single H1 per page)

SEO must be considered during architecture planning, not retrofitted after launch.

Article XIX — AI Agent Constitution

Section 1 — Planning Before Generation **[CRITICAL]**

AI agents must follow this sequence before generating any code:

1. Analyze requirements fully before proposing solutions
2. Define architecture, routes, and component hierarchy
3. Define reusable systems and shared patterns
4. Define dependencies and state flow
5. Validate assumptions explicitly
6. Only then generate implementation code

> **Blind code generation without upfront planning is explicitly prohibited.**

Section 2 — Hallucination Prevention Constitution **[CRITICAL]**

AI agents must NOT:

- invent API endpoints
- fabricate database schemas
- assume unavailable library functions
- reference non-existent routes
- use unsupported syntax

When uncertain about any implementation detail, the agent must explicitly mark the output:

```
[Unverified] — This endpoint may not exist. Confirm with backend team before use.
[Assumed]    — Schema inferred from context. Validate against actual DB.
```

Silence about uncertainty is a failure mode. Explicit uncertainty markers are preferable to confident incorrect output.

Section 3 — Consistency Preservation Constitution **[CRITICAL]**

AI agents must preserve across every code generation session:

- existing naming conventions
- existing folder structures
- component reuse patterns
- design token systems
- architectural patterns already established

Generated code must integrate naturally with the existing codebase, not introduce conflicting patterns.

Section 4 — Pre-Output Validation Checklist

Before finalizing any code output, the AI agent must verify:

Check	Requirement
Imports	All imports reference real, existing modules
Syntax	No syntax errors in generated code
Responsiveness	UI components include responsive handling
Accessibility	ARIA labels, semantic HTML, focus states present
Duplicate Logic	No logic duplicated from existing codebase
Naming	Consistent with project naming conventions
State	State logic consistent with existing store patterns
Security	No secrets, no XSS vectors, inputs sanitized
Hallucinations	No invented APIs, schemas, or functions

Article XX — Documentation Constitution

Every project must include:

- setup instructions
- environment configuration steps
- folder structure overview
- architecture overview
- deployment instructions
- API documentation

Undocumented systems become unmaintainable over time regardless of code quality.

Article XXI — Deployment Constitution

Before any production deployment, verify:

- optimized and minified assets
- all debug logs removed
- environment variables correctly set
- production build stability confirmed
- responsive design tested across breakpoints
- accessibility validated
- SEO metadata present
- security headers configured:
  - `Content-Security-Policy`
  - `Strict-Transport-Security` (HSTS)
  - `X-Frame-Options`
  - `X-Content-Type-Options`

Article XXII — Engineering Governance Constitution

Section 1 — Conflict Resolution Priority

When multiple valid architectural solutions exist, resolve using this priority order:

Priority	Value
1	Security
2	Correctness
3	Maintainability
4	Scalability

Priority	Value
5	Performance
6	Developer convenience

Section 2 — Anti-Overengineering Constitution

Do not create abstractions before patterns stabilize.

```
Avoid: UniversalAbstractFactoryManagerProvider
       without a legitimate, documented scaling need.
```

Premature abstraction creates unnecessary complexity that future developers must reverse.

Section 3 — Proportional Engineering Constitution

Engineering complexity must scale with product complexity.

Projects such as landing pages, MVPs, and internal dashboards may not require large-scale abstractions. Apply patterns that match the actual current and near-term scale of the system.

Final Universal Principle

> *A website is not merely a collection of pages. It is a scalable system of consistent experiences, predictable behaviors, maintainable architecture, and reliable interactions.*

Principle	Outcome
Consistency	creates trust
Predictability	creates maintainability
Scalability	creates longevity
Architecture	determines the future stability of the system
Clarity	is always preferred over chaos